Introduction to Neural Networks

Optimization strategies

Juan Irving Vásquez
jivg.org

Centro de Innovación y Desarrollo Tecnológico en Cómputo
Instituto Politécnico Nacional

October 26, 2026©



Contents

1. Optimization concepts
2. Gradient Descent
3. Stochastic Gradient Descent
4. Including Momentum
5. Learning rate estimation
6. Adaptative gradient

Roadmap

1. Optimization concepts
2. Gradient Descent
3. Stochastic Gradient Descent
4. Including Momentum
5. Learning rate estimation
6. Adaptative gradient

Optimization

- It is to decrease or increase the value of a function $f(x)$ by altering x.
- In deep learning the objective function is called **loss function** or **error function**.

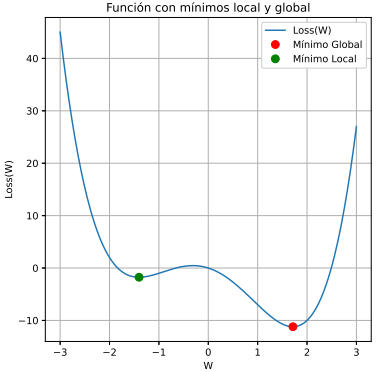
$$\mathcal{L}(W, D),$$

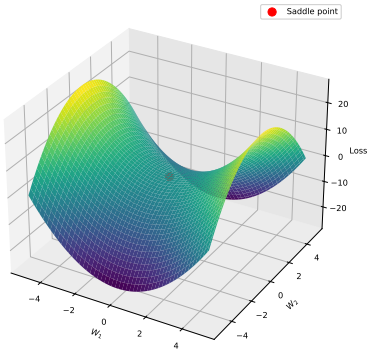
  which is read a the loss of the model for the dataset D using W parameters.
- In general, we want to minimize the Loss to find the best parameters:

$$W^* = \arg\min \mathcal{L}(W, D) \tag{1}$$

Critical points

- Points where $\mathcal{L}'(W, D) = 0$ (derivate) are stationary points (SP):
  - A local minimum is an SP where $\mathcal{L}(W, D)$ is lower than its neighbors
  - Nor maximum or minimum SP are saddle points
  - The absolute lowest value of $\mathcal{L}(W, D)$ is a global minimum





Goal of Optimization vs. Deep Learning

- Optimization aims to **minimize the loss function** — usually computed on the training data.
- Deep learning aims to **find a model that generalizes well** to unseen data.
- Training error $\neq$ generalization error.
- Minimizing training loss alone may lead to **overfitting**.
- Successful learning balances optimization efficiency and generalization ability.

Generalization and Risk Minimization

- **Empirical Risk:** Average loss over the training dataset [1].

$$\hat{R}(f) = \frac{1}{n} \sum_{i=1}^{n} \mathcal{L}(\Phi(x_i), y_i) \tag{2}$$

- **Expected Risk (True Risk):** Expected loss over the unknown data distribution.

$$R(f) = \mathbb{E}_{(x,y)\sim\mathcal{D}}[\mathcal{L}(\Phi(x), y)] \tag{3}$$

- Deep learning seeks to minimize expected risk, but we can only optimize empirical risk.
- **Generalization** refers to how close these two risks are.

[1] $\theta(x)$ indicates the prediction of the network; y the true label

Empirical risk optimization[2]

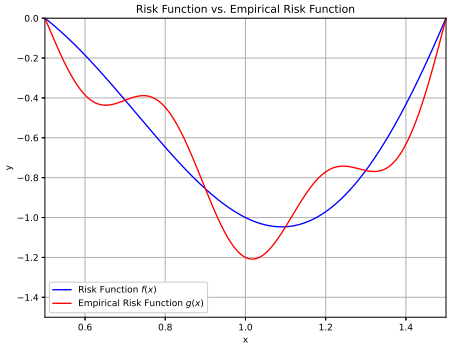


Figure: Deep learning seeks to minimize risk, but we can only optimize empirical risk.

[2] Zhang, Aston, et al. Dive into deep learning. Cambridge University Press, 2023.

Roadmap

1. Optimization concepts
2. Gradient Descent
3. Stochastic Gradient Descent
4. Including Momentum
5. Learning rate estimation
6. Adaptative gradient

Gradient descent

- Optimization algorithm that iteratively updates the parameters of a function.

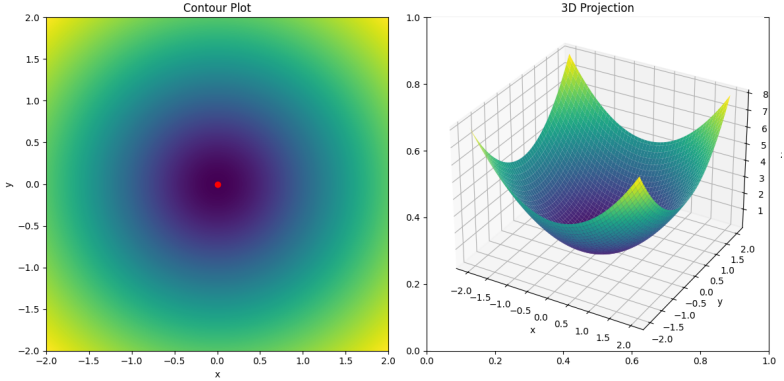
$$W_{\text{new}} = W_{\text{old}} - \eta \nabla_W \mathcal{L}(W, D) \tag{4}$$

where $\mathcal{L}$ is the loss function, η the learning rate, W the parameters of the model and D the dataset.

Example

Spheric function.

$$f(x, y) = x^2 + y^2 \tag{5}$$



Example



Figure: Solution to the sphere funcion.

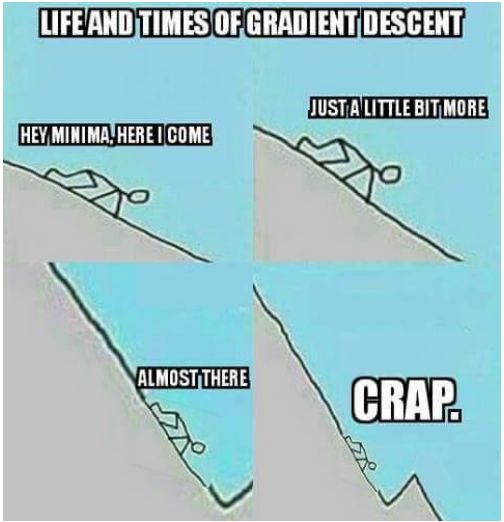


Figure: True story

Roadmap

1. Optimization concepts
2. Gradient Descent
3. Stochastic Gradient Descent
4. Including Momentum
5. Learning rate estimation
6. Adaptative gradient

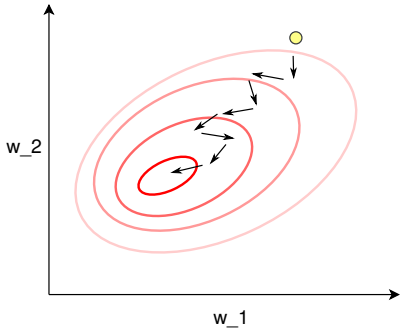
Stochastic Gradient Descent (SGD)

- For large datasets. We update the weights using a random set of tuples (x_i, y_i), **Minibatch**. The algorithm achieves linear convergence [3]

[3] Bottou L. (2012) Stochastic Gradient Descent Tricks. In: Montavon G., Orr G.B., Müller KR. (eds) Neural Networks: Tricks of the Trade. Lecture Notes in Computer Science, vol 7700. Springer, Berlin, Heidelberg..

Stochastic Gradient Descent (SGD)

- For large datasets. We update the weights using a random set of tuples (x_i, y_i), **Minibatch**. The algorithm achieves linear convergence [3]



[3] Bottou L. (2012) Stochastic Gradient Descent Tricks. In: Montavon G., Orr G.B., Müller KR. (eds) Neural Networks: Tricks of the Trade. Lecture Notes in Computer Science, vol 7700. Springer, Berlin, Heidelberg..

Batch Size

- The batch size determines how many training examples are used in one iteration.
- Typical values: 32, 64, 128. Smaller batches add noise but can help escape local minima.
- Large batches converge faster but may get stuck in sharp minima.

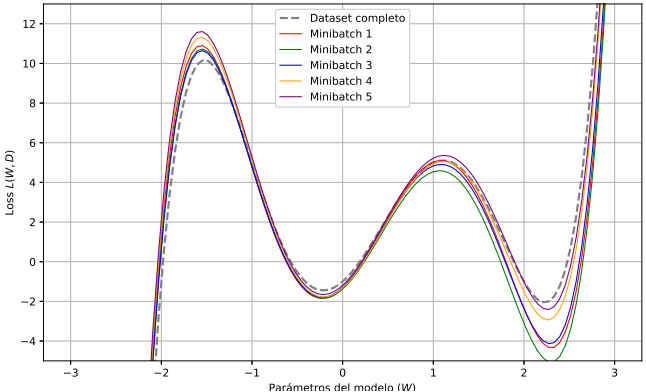


Figure: Loss function for several datasets

Batch Normalization

- Normalizes inputs to each layer to have mean 0 and variance 1.
- Reduces internal covariate shift.
- Improves training stability and allows higher learning rates.

So far so good

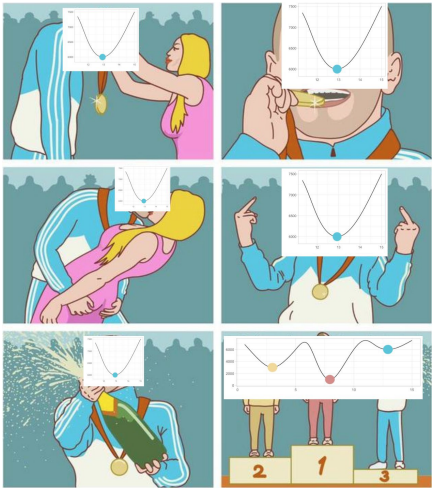


Figure: Stochastic gradient descent

Roadmap

1. Optimization concepts
2. Gradient Descent
3. Stochastic Gradient Descent
4. Including Momentum
5. Learning rate estimation
6. Adaptative gradient

SGD - Momentum

- SGD may over sigzag
- The momentum can improve the efficiency.

$$M_t = \beta M_{t-1} + (1 - \beta)\nabla_w \mathcal{L}(W, X, Y) \tag{6}$$

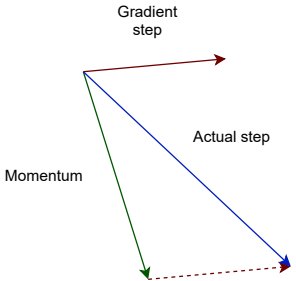
$$W_t = W_{t-1} - \eta M_t \tag{7}$$

SGD - Momentum

- SGD may over sigzag
- The momentum can improve the efficiency.

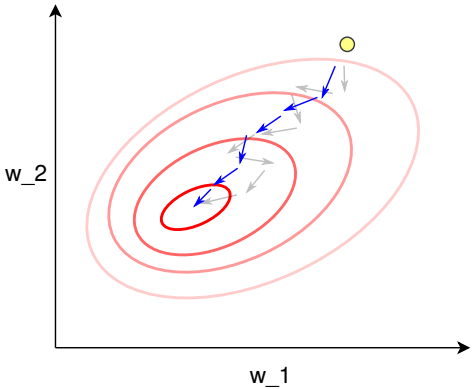
$$M_t = \beta M_{t-1} + (1-\beta)\nabla_w \mathcal{L}(W, X, Y) \tag{6}$$

$$W_t = W_{t-1} - \eta M_t \tag{7}$$



- Usually $\beta = 0.9$

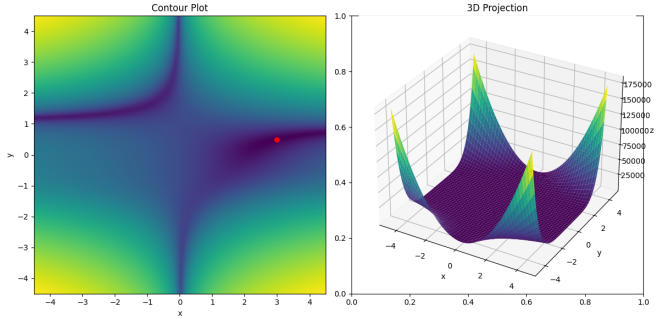
SGD - Momentum (2)



Example: Beale's function

- La función de Beale es una función de prueba comúnmente utilizada en la optimización para evaluar el rendimiento de algoritmos en problemas no convexos.

$$f(x, y) = (1.5 - x + xy)^2 + (2.25 - x + xy^2)^2 + (2.625 - x + xy^3)^2 \tag{8}$$



Example

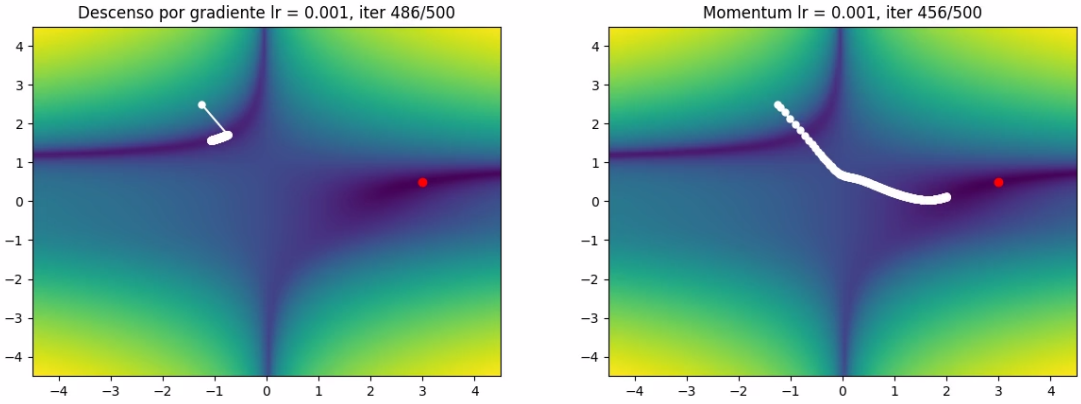


Figure: Performance of the optimization methods

Roadmap

1. Optimization concepts
2. Gradient Descent
3. Stochastic Gradient Descent
4. Including Momentum
5. Learning rate estimation
6. Adaptative gradient

What about the learning rate?



Warm-up Strategy

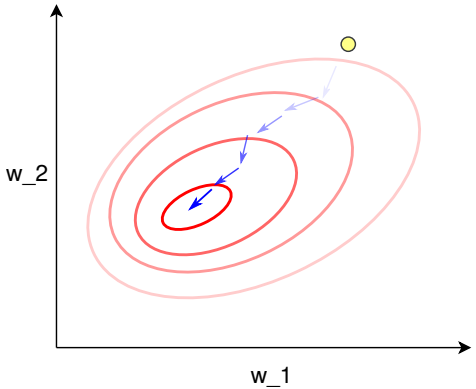
- Warm-up gradually increases the learning rate during the initial training phase [4].
- Helps avoid instability due to large initial gradient updates.
- Often used with adaptive optimizers such as Adam.
- Linear warm-up:

$$\eta_t = \eta_{max} \cdot \frac{t}{T_{warmup}} \quad \text{if } t \leq T_{warmup} \tag{9}$$

[4] Kalra, Dayal Singh, and Maissam Barkeshli. "Why warmup the learning rate? underlying mechanisms and improvements." Advances in Neural Information Processing Systems 37 (2024): 111760-111801.

Learning Rate Decay

- As we aproximate to the minimum the step may overshoot the target.



Learning Rate Decay (2)

- A solution is to apply a learning rate decay.

$$\eta = \eta_0 \left(\frac{1}{1 + id} \right) \tag{10}$$

where η_0 is the starting l.r., i is the epoch and d is the decay hyperparámeter.

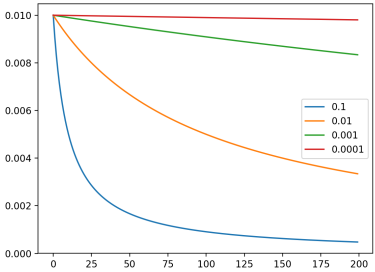


Figure: Effect of the decay (d) on the learning rate.

Roadmap

1. Optimization concepts
2. Gradient Descent
3. Stochastic Gradient Descent
4. Including Momentum
5. Learning rate estimation
6. Adaptative gradient

Motivation



Figure: Which are infrecuent features?

Optimizing for infrecuent features: Adagrad

- The core problem: The learning rate decays too quickly for infrequent features (which need more updates) or too slowly for common features.
- First add gradient clippling.
- Adagrad (Duchi et al., 2011) addresses this by providing an adaptive, per-parameter learning rate.
- Instead of using a crude update counter (n_i), Adagrad scales the learning rate for each parameter using the accumulated *sum of the squares of its past gradients*.
- This mechanism automatically reduces the learning rate for parameters with large, frequent gradients and maintains a higher rate for parameters with small, infrequent gradients.

Gradient Clipping

- Prevents exploding gradients during backpropagation.
- Clip gradients to a threshold value:

$$\text{if } \|g\| > \tau, \quad g \leftarrow \tau \cdot \frac{g}{\|g\|} \tag{11}$$

- Especially useful in RNNs and deep networks.

Gradient Clipping

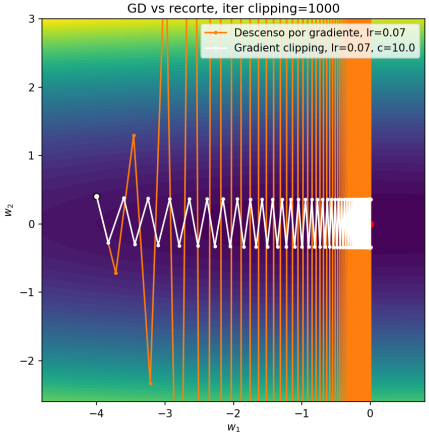


Figure: Example of gradient clipping implementation.

AdaGrad [5]

Accumulation of gradients

- Adapts gradient over time using an accumulation of the historical gradients
- Initialization

$$r_0 = 0 \tag{12}$$

- Accumulate the square of the gradients

$$r_i = r_{i-1} + g_i \odot g_i \tag{13}$$

- Update

$$w_i = w_{i-1} - \frac{\eta}{k \oplus \sqrt{r_i}} \odot g_i \tag{14}$$

where $k \approx 10^{-7}$

[5] Duchi, John, Elad Hazan, and Yoram Singer. "Adaptive Subgradient Methods for Online Learning and Stochastic Optimization." Journal of Machine Learning Research 12.Jul (2011): 2121-2159

AdaGrad

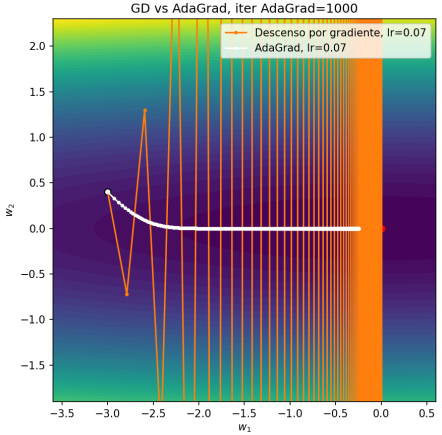


Figure: Example of the AdaGrad implementation

RMSProp [6]

- RMSprop (Root Mean Square Propagation) improves over Adagrad.
- Adagrad + momentum
- Discards old measurements with ρ

$$r_i = \rho r_{i-1} + (1-\rho) g_i \odot g_i \tag{15}$$

where ρ determines how long we keep old gradients

- Update

$$w_i = w_{i-1} - \frac{\eta}{\sqrt{k \oplus r_i}} \odot g_i \tag{16}$$

where $k \approx 10^{-7}$

[6] Tieleman, T., & Hinton, G. (2012). Rmsprop: Divide the gradient by a running average of its recent magnitude. Neural networks for machine learning. 17, 6.

Adam [7]

- Exponentially weighted moving average of the gradient (first momentum)

$$m_i = \beta_1 m_{i-1} + (1 - \beta_1) g_i \tag{17}$$

- Moving average of the historical gradients (second momentum)

$$v_i = \beta_2 v_{i-1} + (1 - \beta_2) g_i \odot g_i \tag{18}$$

- Correction

$$\tilde{m}_i = \frac{m_i}{1 - \beta_1^i}$$

$$\tilde{v}_i = \frac{v_i}{1 - \beta_2^i}$$

- Update

$$w_i = w_{i-1} - \frac{\eta}{k \oplus \sqrt{\tilde{v}_i}} \tilde{m}_i$$

[7] Adam, Kingma DP Ba J. "A method for stochastic optimization." arXiv preprint arXiv:1412.6980 1412.6 (2014).

Optimizer selection

- Consider them part of the design of experiment
- Currently, the main advances have been achieved by designing easy to train networks, instead of better optimizers
- Use Adam and improve the architecture

Extra readings

- **Chapter 4: Numerical Optimization.** Goodfellow, I., Bengio, Y., Courville, A., & Bengio, Y. (2016). Deep learning (Vol. 1). Cambridge: MIT press.
- **Chapter 6: Beyond Gradient Descent.** Buduma, Nithin, Nikhil Buduma, and Joe Papa. Fundamentals of deep learning: Designing next-generation machine intelligence algorithms. " O'Reilly Media, Inc.", 2022.